Procesos

Unidad didáctica 1

Programación de servicios y procesos, 2º DAM. Alex Quesada, IES Tierno Galván

Índice

1. Programas, procesos y servicios
2. El sistema operativo y la ejecución de procesos
3. Hilos
4. Programación concurrente, paralela y distribuida
5. Herramientas de monitorización
6. Crear y controlar procesos desde Java
7. Comunicación entre procesos
8. Sincronización entre procesos
9. Aplicaciones multiproceso

01

Programas, procesos y servicios

Qué hay en marcha en un ordenador

Programa, ejecutable, librería y proceso

Concepto	Qué es	Ejemplo
Programa	Instrucciones que resuelven una tarea (pasivo)	`ContadorPrimos.java`
Ejecutable	Programa en un formato que el SO sabe cargar (pasivo)	`notepad.exe`, `java.exe`
Librería	Código compilado para que lo usen otros programas (pasivo)	`kernel32.dll`, un `.jar`
Proceso	**Programa en ejecución**, con memoria, estado y recursos (activo)	Cada ventana del Bloc de notas

El caso de Java

- Un `.class` contiene *bytecode*: lo entiende la **JVM**, no el procesador
- El ejecutable es `java.exe`: la JVM carga las clases y las ejecuta
- `java ContadorPrimos 1 1000` crea un proceso de `java.exe`, que ejecuta `ContadorPrimos`
- En el Administrador de tareas todos los programas Java se llaman `java.exe`
- Para saber qué clase ejecuta cada uno: `jps` (apartado 5)

Elementos de un proceso

- **PID**: su identificador. También guarda el de su padre (**PPID**)
- **Memoria propia**, aislada: código, datos (`static`), montículo (objetos) y pila (variables locales)
- **Estado y contexto**: listo, en ejecución, bloqueado... y los registros de la CPU
- **Recursos**: ficheros abiertos, conexiones, dispositivos
- **Entorno**: usuario, variables de entorno, directorio de trabajo y prioridad
- **Flujos estándar**: entrada, salida y salida de error (`System.in`, `System.out`, `System.err`)

Creación y finalización de procesos

- Todo proceso lo crea otro, su **padre**: se forma un **árbol de procesos**
- Unix: `fork()` duplica el proceso y `exec()` cambia su programa. Windows: `CreateProcess`
- Java: `ProcessBuilder.start()` usa el mecanismo de cada sistema (apartado 6)
- Termina al acabar su programa, por un error o porque otro proceso lo termina
- Devuelve un **código de salida**: `0` si todo ha ido bien; otro valor, si ha habido un error

Servicios y demonios

Un **servicio** es un proceso en segundo plano, sin interfaz, que ofrece una función a otros programas.

Windows

- Consola `services.msc`
- Pestaña *Servicios* del Administrador de tareas
- `sc query Spooler`
- Inicio automático, manual o deshabilitado

Linux: demonios

- Su nombre suele acabar en *d*: `sshd`, `crond`
- `systemctl status ssh`
- `systemctl start`, `stop` y `enable`

Un programa está en el disco y no hace nada. Un proceso es ese programa en ejecución.

Cada proceso tiene su PID, su memoria aislada, su estado y sus recursos.

A01.1 (1/2). Procesos y servicios en tu equipo

1. Administrador de tareas (Ctrl+Mayús+Esc), pestaña **Detalles**: ¿cuántos procesos hay? Anota el nombre y el PID de tres
2. Abre **dos** `cmd`: ¿cuántos `cmd.exe` hay?, ¿tienen el mismo PID?, ¿cuántos ficheros `cmd.exe` hay en el disco?
3. Añade las columnas **Línea de comandos** y **Subprocesos**: ¿qué muestran para un `cmd.exe`?
4. `tasklist /fi "IMAGENAME eq cmd.exe"` y cierra el **otro** `cmd` con `taskkill /pid <PID>`. ¿Y con `/f`?

> **Tiempo:** 30 min, individual. **Entrega:** no se entrega; se pone en común al final de la sesión.

A01.1 (2/2). Procesos y servicios en tu equipo

5. Ejecuta en IntelliJ un programa que espere a que escribas algo (o el ejemplo 3.3): ¿con qué nombre aparece?, ¿cómo sabrías qué clase ejecuta?
6. `services.msc`: nombre, estado, tipo de inicio y función de dos servicios. Comprueba la cola de impresión con `sc query Spooler`
7. Resume en cuatro líneas qué es un programa, un ejecutable, un proceso y un servicio, con un ejemplo de tu equipo

> **Tiempo:** incluido en los 30 min. **Entrega:** no se entrega; se pone en común al final de la sesión.

02

El sistema operativo y la ejecución de procesos

Más procesos que núcleos: cómo avanzan todos

Multitarea y cambio de contexto

- Un núcleo ejecuta **un solo proceso** en cada instante
- **Multitarea**: el SO alterna los procesos cada pocos milisegundos (el **cuanto**)
- **Expulsiva**: el SO quita la CPU al acabar el cuanto (Windows, Linux, macOS)
- **Cooperativa**: cada proceso cede la CPU cuando quiere (Windows 3.x)
- **Cambio de contexto**: guardar en el PCB los registros del que sale y cargar los del que entra
- Dura microsegundos sin trabajo útil: un cuanto muy corto hace perder tiempo

Estados de un proceso

Estado	Significado
Nuevo	Se está creando: el SO prepara su PCB y su memoria
Listo	Puede ejecutarse; solo espera a que le den la CPU
En ejecución	Sus instrucciones se ejecutan en un núcleo
Bloqueado	Espera algo: el disco, un dato de la red o de una tubería, que termine un hijo…
Terminado	Ha acabado; el SO libera sus recursos

Cambios de estado

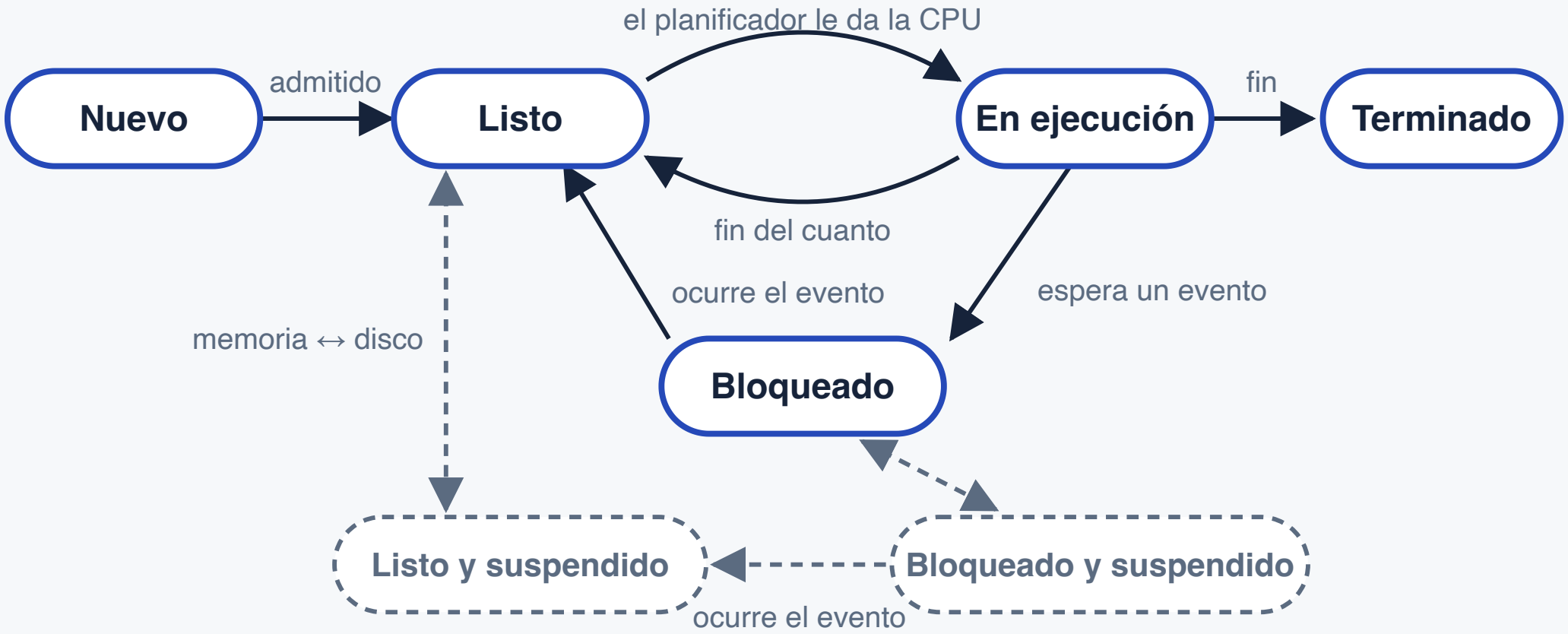
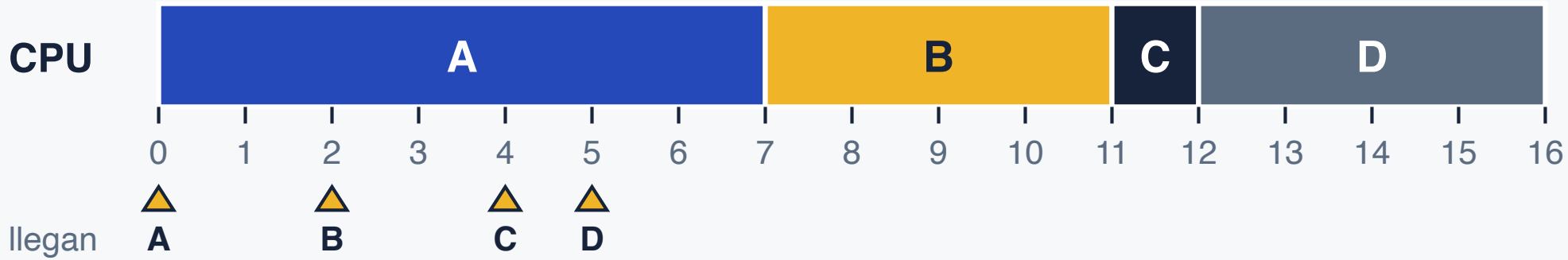


Figura 1. Los estados suspendidos son los de un proceso que se ha sacado de la memoria principal al disco

Planificación: quién elige y cómo se mide

- El **planificador** elige qué proceso listo se ejecuta; el **despachador** hace el cambio de contexto
- **No expulsiva**: conserva la CPU hasta que termina o se bloquea. **Expulsiva**: se la pueden quitar
- **Retorno** = fin – llegada
- **Espera** = retorno – ráfaga (el tiempo en la cola de listos)
- **Respuesta** = primera vez en la CPU – llegada
- Un buen algoritmo es equitativo y evita la **inanición**: esperar siempre porque otros pasan delante

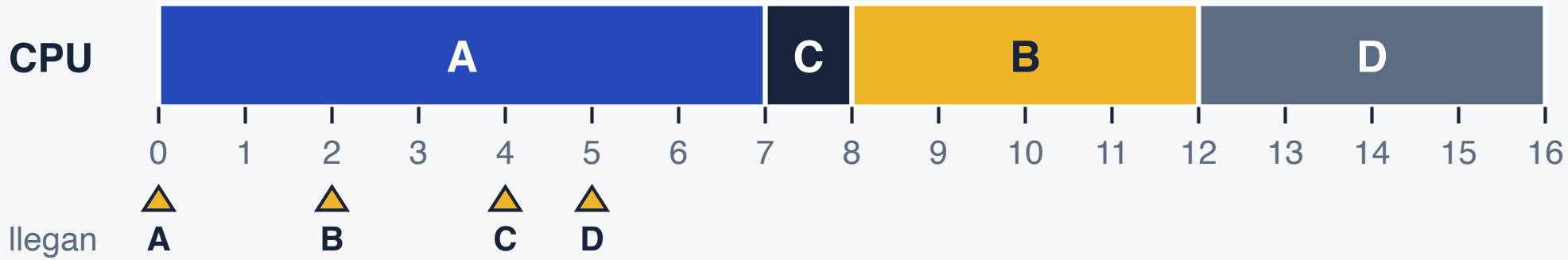
FIFO: por orden de llegada



Proceso	Llegada	Ráfaga	Prioridad	Fin	Retorno	Espera
A	0	7	3	7	7	0
B	2	4	1	11	9	5
C	4	1	4	12	8	7
D	5	4	2	16	11	7
Media					**8,75**	**4,75**

Figura 2. FIFO (no expulsivo): cada proceso hasta el final, en orden de llegada

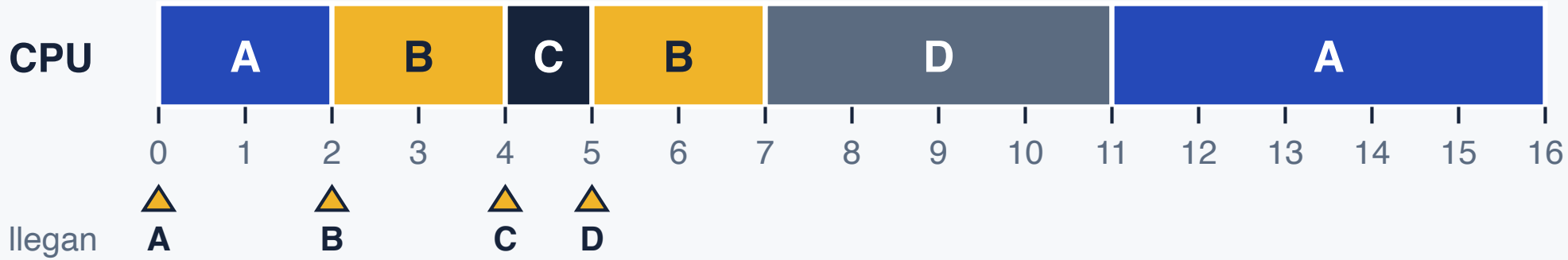
SJF: el más corto de los que han llegado



Proceso	Llegada	Ráfaga	Prioridad	Fin	Retorno	Espera
A	0	7	3	7	7	0
B	2	4	1	12	10	6
C	4	1	4	8	4	3
D	5	4	2	16	11	7
Media					**8,00**	**4,00**

Figura 3. SJF (no expulsivo): en el instante 7 va C (ráfaga 1); B y D empatan a 4 y va B, que llegó antes

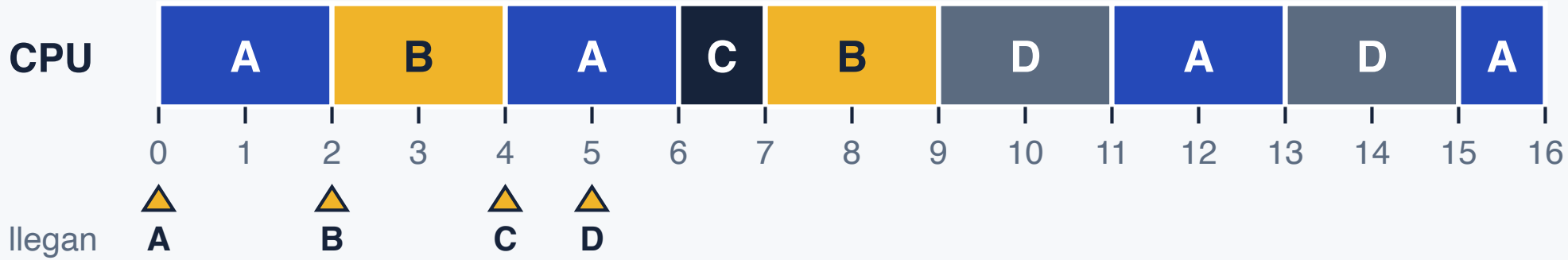
SRT: el que menos tiempo le queda



Proceso	Llegada	Ráfaga	Prioridad	Fin	Retorno	Espera
A	0	7	3	16	16	9
B	2	4	1	7	5	1
C	4	1	4	5	1	0
D	5	4	2	11	6	2
Media					**7,00**	**3,00**

Figura 4. SRT (expulsivo): B expulsa a A en el instante 2 y C expulsa a B en el 4; A termina el último

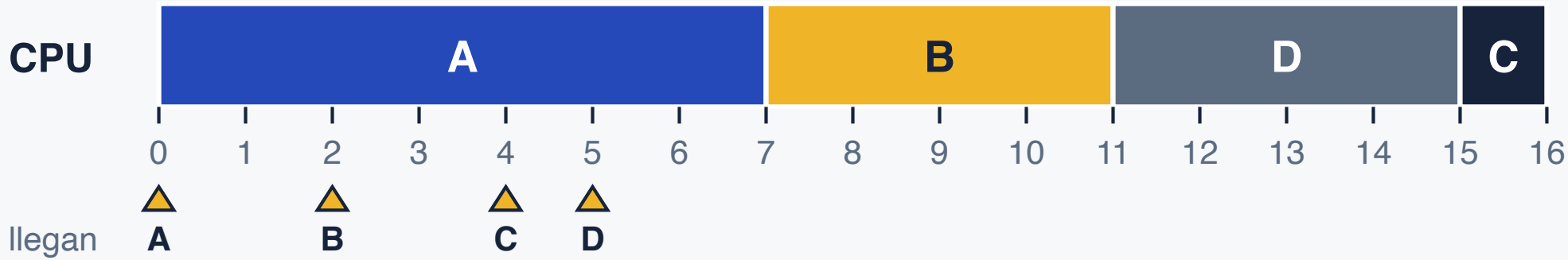
Round Robin (cuanto = 2)



Proceso	Llegada	Ráfaga	Prioridad	Fin	Retorno	Espera
A	0	7	3	16	16	9
B	2	4	1	9	7	3
C	4	1	4	7	3	2
D	5	4	2	15	10	6
Media					**9,00**	**5,00**

Figura 5. Round Robin (expulsivo): cola circular; quien llega entra en la cola antes que el expulsado

Por prioridades (menor número, más prioridad)



Proceso	Llegada	Ráfaga	Prioridad	Fin	Retorno	Espera
A	0	7	3	7	7	0
B	2	4	1	11	9	5
C	4	1	4	16	12	11
D	5	4	2	15	10	6
Media					**9,50**	**5,50**

Figura 6. Prioridades (no expulsivo): C necesita 1 unidad y termina el último, por su prioridad 4

Comparación de los algoritmos

Algoritmo	Expulsivo	Retorno medio	Espera media	Respuesta media	Inanición
FIFO	No	8,75	4,75	4,75	No
SJF	No	8,00	4,00	4,00	Sí, los largos
SRT	Sí	7,00	3,00	0,50	Sí, los largos
Round Robin (q = 2)	Sí	9,00	5,00	1,50	No
Prioridades	No	9,50	5,50	5,50	Sí, los poco prioritarios

Un proceso bloqueado nunca pasa directamente a ejecución: vuelve a listo.

Y mientras está bloqueado, no consume CPU.

A01.2 (1/2). Planificación de procesos

Un núcleo; el cambio de contexto no cuesta. Empate: antes el que llegó antes.

Proceso	Llegada	Ráfaga	Prioridad
A	0	5	3
B	1	3	1
C	2	8	4
D	4	2	2
E	6	4	2

> **Tiempo:** 40 min, individual. **Entrega:** se corrige en la pizarra; lo que falte, en casa.

A01.2 (2/2). Planificación de procesos

1. **FIFO**: Gantt; fin, retorno, espera y respuesta de cada proceso; medias
2. **SJF** (no expulsivo)
3. **SRT**: expulsa solo si su ráfaga es **menor** que lo que le queda al otro
4. **Round Robin**, cuanto 2: quien llega entra antes que el expulsado
5. En grupo: ¿cuál espera menos y por qué no se usa tal cual? ¿Cuál responde mejor? ¿Quién sale peor parado en SJF y SRT?
6. **Ampliación:** prioridades no expulsivo

> **Tiempo:** incluido en los 40 min. **Entrega:** se corrige en la pizarra.

03

Hilos

Varias secuencias de ejecución dentro de un proceso

Qué es un hilo

Un **hilo** es una secuencia de instrucciones dentro de un proceso que el SO planifica por separado.

- Todo proceso tiene al menos uno: el **hilo principal** (en Java, el que ejecuta `main`)
- Con varios hilos, un proceso hace varias cosas a la vez: un navegador descarga en uno y atiende el ratón en otro
- El planificador reparte **hilos** entre los núcleos: lo del apartado 2 vale para cada hilo
- Microsoft, en español, llama «subproceso» al hilo: no es un proceso hijo

Qué comparten los hilos de un proceso



Figura 7. Los hilos comparten la memoria y los recursos del proceso; cada uno tiene su pila y sus registros

Procesos frente a hilos

Procesos

- Memoria propia, aislada
- Crearlos y cambiar de contexto es **costoso**
- Se comunican a través del SO: tuberías, ficheros, red...
- Si uno falla, los demás siguen
- En Java, `ProcessBuilder` (UD 1)

Hilos

- Comparten la memoria del proceso
- Crearlos y cambiar de contexto es **ligero**
- Se comunican con variables y objetos compartidos
- Un fallo grave puede tumbar el proceso
- En Java, `Thread` (UD 2)

Ejemplo 3.3: `HilosDeLaJvm`

```java
import java.util.Set;

public class HilosDeLaJvm {

    public static void main(String[] args) {
        System.out.println("PID del proceso: " + ProcessHandle.current().pid());
        System.out.println("Hilo que ejecuta main: " + Thread.currentThread().getName());
        System.out.println();

        Set<Thread> hilos = Thread.getAllStackTraces().keySet();
        System.out.println("Hilos vivos en este proceso: " + hilos.size());
        for (Thread hilo : hilos) {
            String tipo = hilo.isDaemon() ? "demonio" : "normal";
            System.out.printf("  %-22s %-8s %s%n", hilo.getName(), tipo, hilo.getState());
        }
    }
}
```

Los hilos de un proceso comparten su memoria; los procesos, no.

Por eso los hilos se comunican fácilmente y casi siempre necesitan sincronizarse (UD 2).

04

Programación concurrente, paralela y distribuida

Estructurar, terminar antes o repartir entre máquinas

Programación concurrente

Dos tareas son **concurrentes** si sus ejecuciones se solapan: una empieza antes de que acabe la otra.

- **No determinismo**: el orden lo decide el planificador y cambia en cada ejecución
- Las tareas necesitan **comunicarse y sincronizarse** (apartados 7 y 8)
- **Difícil de depurar**: los errores dependen del orden, aparecen solo algunas veces y pueden desaparecer al depurar

Concurrencia no es paralelismo

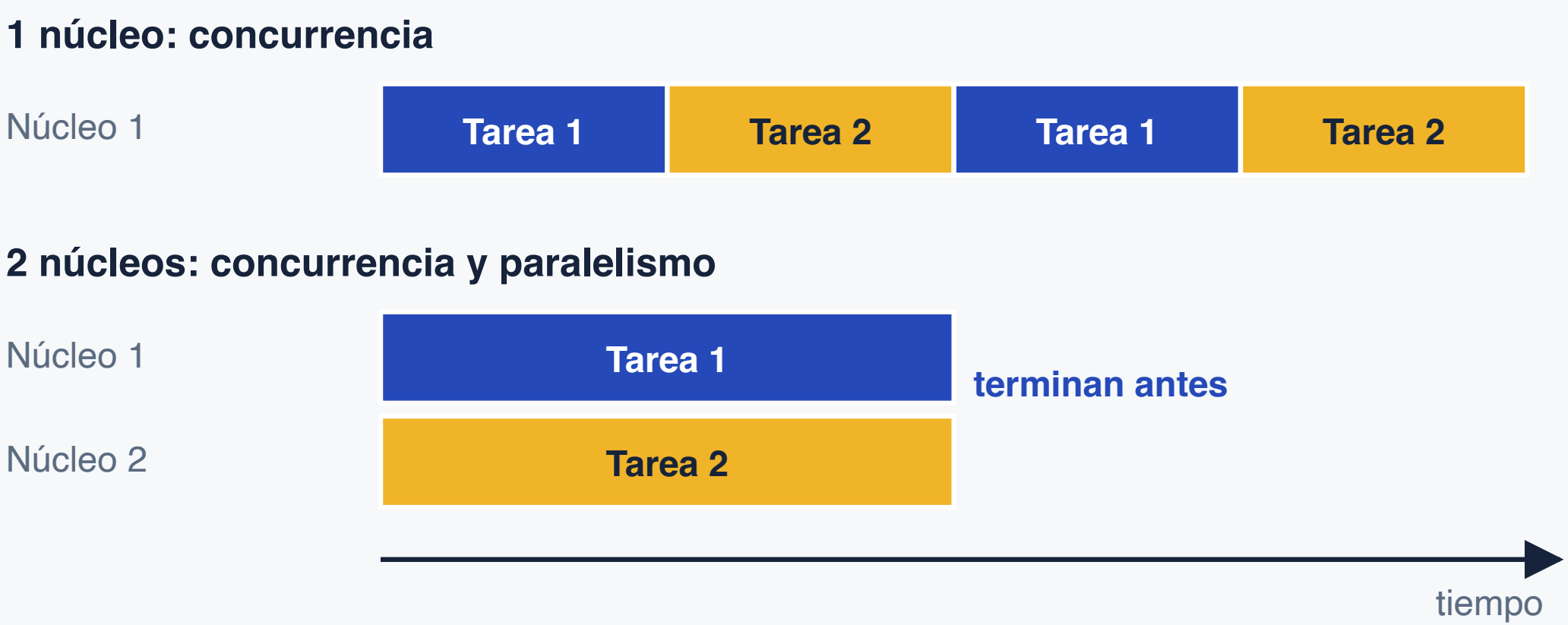


Figura 8. Concurrencia: varias tareas en curso. Paralelismo: a la vez, en núcleos distintos

Ámbitos de aplicación

Ámbito	Qué aporta	Ejemplos
Cálculo intensivo	Terminar antes repartiendo entre núcleos	Vídeo, simulaciones, compilar
Interfaces	Que la interfaz no se congele	App Android que descarga datos
Servidores	Atender a muchos clientes a la vez	Web, bases de datos (UD 3 y 4)
Entrada y salida	Usar la CPU mientras se espera al disco o la red	Navegador que descarga varios recursos
Sistemas concurrentes	Modelar actividades simultáneas	Control, videojuegos, el propio SO

Programación paralela

Varias tareas **a la vez**, en varios núcleos de **una misma máquina**, para terminar antes.

Ventajas

- Menos tiempo en los problemas que se pueden descomponer
- Problemas más grandes en el mismo tiempo
- Aprovecha los núcleos del equipo

Inconvenientes

- No todo se puede paralelizar
- Repartir y reunir cuesta tiempo
- Problemas de sincronización (apartado 8)
- La mejora tiene un techo

Ley de Amdahl

Si una fracción *p* del programa se puede paralelizar, la aceleración máxima con *N* núcleos es:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}$$

p	N = 2	N = 4	N = 10	Máximo
50 %	1,33	1,60	1,82	2
90 %	1,82	3,08	5,26	10

Programación distribuida

Varios **ordenadores** conectados por una **red** que colaboran enviándose **mensajes**: no comparten memoria ni reloj.

Ventajas

- Escala añadiendo máquinas
- Tolera fallos: si una cae, otra sigue
- Comparte recursos entre lugares distintos
- Servidores cerca de los usuarios

Inconvenientes

- La red es lenta y puede fallar
- Datos coherentes: difícil
- Más complejo de programar
- Hay que proteger los datos (UD 5)

Concurrente, paralela y distribuida

Aspecto	Concurrente	Paralela	Distribuida
Idea	Varias tareas en curso a la vez	Varias en el mismo instante	Varias máquinas que colaboran
Objetivo	Estructurar, responder, atender a varios	Terminar antes	Escalar, tolerar fallos
Hardware	Uno o varios núcleos	Varios núcleos, una máquina	Varios ordenadores en red
Comunicación	Memoria compartida o mensajes	Memoria compartida	Mensajes por la red
En el módulo	Todo el módulo	UD 1 y UD 2	UD 3 y UD 4

La concurrencia es la estructura: varias tareas en curso. El paralelismo es la ejecución: a la vez, en varios núcleos.

La paralela usa una sola máquina; la distribuida, varias conectadas en red.

A01.3 (1/2). ¿Concurrente, paralela o distribuida?

1. **Clasificad** los 8 casos del enunciado como concurrente sin paralelismo, paralelo o distribuido, y justificad cada uno en una frase
2. **¿Procesos o hilos?** Decidid en los 3 casos del enunciado y explicad por qué
3. En el ejemplo 3.3, ¿cuántos procesos y cuántos hilos hay? ¿Quién crea esos hilos?

> **Tiempo:** 30 min con la parte 3, en grupo. **Entrega:** no se entrega; cada grupo expone uno de los casos.

A01.3 (2/2). Ley de Amdahl

Un programa tarda 100 s en un núcleo, y el 80 % de su trabajo se puede repartir.

1. ¿Qué aceleración tendrá con 4 núcleos? ¿Cuánto tardará?
2. ¿Y con 16 núcleos?
3. ¿Cuál es la aceleración máxima, tenga los núcleos que tenga? ¿Por qué?
4. El ejemplo 9.1 reparte un cálculo entre 10 procesos y la aceleración es 4,03. Dad dos razones por las que no llega a 10

> **Tiempo:** incluido en los 30 min. **Entrega:** no se entrega; se pone en común.

05

Herramientas de monitorización

Ver, vigilar y terminar procesos

Administrador de tareas (Ctrl+Mayús+Esc)

- **Procesos**: aplicaciones y procesos en segundo plano, con su CPU y su memoria
- **Rendimiento**: uso de la CPU; con el botón derecho, una gráfica por procesador lógico
- **Detalles**: un proceso por fila, con su **PID**; se añaden las columnas *Línea de comandos* y *Subprocesos*
- Botón derecho: **finalizar tarea**, **finalizar árbol de procesos**, **establecer prioridad**
- **Servicios**: los servicios del sistema y su estado
- Los programas Java aparecen todos como `java.exe` (o `javaw.exe`)

Comandos de Windows

Comando	Qué hace
`tasklist`	Lista los procesos con su PID y su memoria
`tasklist /fi "IMAGENAME eq java.exe"`	Solo los que cumplen el filtro
`taskkill /pid 1234`	Pide al proceso que termine
`taskkill /pid 1234 /f`	Lo termina a la fuerza
`taskkill /im java.exe /f /t`	Todos los `java.exe` y sus hijos (`/t`)
`start /low java ContadorPrimos 1 100000000`	Lanza un programa con prioridad baja

Prioridades y herramientas de Linux

Windows

- Clases: Tiempo real, Alta, Por encima de lo normal, **Normal**, Por debajo de lo normal y Baja
- Los hijos heredan la del padre
- Tiempo real puede dejar sin CPU al propio sistema

Linux y macOS

- `ps -ef`, `top` y `kill <PID>`
- Valor **nice**: de –20 (máxima) a 19 (mínima); por defecto, 0
- `nice -n 10 programa` y `renice`
- Solo el administrador puede subir la prioridad

`jps` y `jcmd`: los procesos Java

- Vienen con el JDK y funcionan igual en Windows, Linux y macOS
- `jps -l`: los procesos Java con su PID y su clase principal
- `jcmd <PID> Thread.print`: todos los hilos del proceso y qué hace cada uno
- `jcmd <PID> VM.uptime`: cuánto tiempo lleva en marcha

```
$ jps -l
15617 jdk.jcmd/sun.tools.jps.Jps
15615 ContadorPrimos
```

06

Crear y controlar procesos desde Java

`ProcessBuilder`, `Process` y `ProcessHandle`

La API de procesos (`java.lang`)

Tipo	Para qué sirve	Métodos principales
`ProcessBuilder`	Preparar y lanzar un proceso	`command`, `directory`, `environment`, `redirect...`, `inheritIO`, `start`, `startPipeline`
`Process`	Controlar un hijo ya lanzado	`pid`, `inputReader`, `errorReader`, `outputWriter`, `waitFor`, `exitValue`, `isAlive`, `destroy`
`ProcessHandle`	Consultar cualquier proceso del sistema	`current`, `of`, `parent`, `children`, `descendants`, `allProcesses`, `info`

Cinco pasos para lanzar un proceso

1. Crear un `ProcessBuilder` con el **comando**: cada argumento en su propia cadena
2. Configurarlo si hace falta: carpeta de trabajo, entorno, redirecciones
3. `start()` crea el proceso y devuelve un `Process` (si el programa no existe, `IOException`)
4. Comunicarse con el hijo por sus flujos (apartado 7)
5. Esperar con `waitFor()` y comprobar su código de salida (apartado 8)

> Nada de `Runtime.exec("comando argumentos")`: está obsoleto desde Java 18.

Programas externos y portabilidad

- `ProcessBuilder` puede lanzar cualquier ejecutable: `git`, `ffmpeg`, un programa en C...
- Pero el código **deja de ser portable**: `dir` en Windows (una orden de `cmd`), `ls` en Linux
- Lanza el programa **sin intérprete de comandos**: no entiende `|`, `>`, `*` ni las comillas
- Tuberías y redirecciones, con los métodos de `ProcessBuilder` (apartado 7)
- En este módulo, **los hijos son siempre programas Java** lanzados con la misma JVM

Lanzar otro programa Java

El comando es el mismo que se escribiría en la terminal:

```
java -cp <classpath> <Clase> <argumentos...>
```

- `java.home`: la carpeta de la JVM del padre; el ejecutable está en `bin/java`
- `java.class.path`: el *classpath* del padre, para que el hijo encuentre sus clases
- Funciona igual desde IntelliJ y desde la terminal, en Windows, Linux y macOS

El hijo: `ContadorPrimos`

Está pensado para que lo lance otro programa. Este es su **contrato**:

Aspecto	Contrato
Uso	`java ContadorPrimos <desde> <hasta>`
Salida estándar	**Solo** el número de primos del intervalo, por ejemplo `78498`
Salida de error	Los mensajes de error
Código de salida	`0` si todo va bien; `1` si faltan argumentos; `2` si no son números

Ejemplo 6.3: Lanzador (1/2)

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class Lanzador {
    public static void main(String[] args) throws IOException, InterruptedException {
        List<String> comando = comandoJava("ContadorPrimos", "1", "1000000");
        System.out.println("Comando: " + comando);
        ProcessBuilder pb = new ProcessBuilder(comando);
        pb.inheritIO();
        Process hijo = pb.start();
        System.out.println("Hijo lanzado con PID " + hijo.pid() + "; su salida es:");
        int codigo = hijo.waitFor();
        System.out.println("El hijo ha terminado con código " + codigo);
    }
    // comandoJava(...): en la diapositiva siguiente
}
```

Ejemplo 6.3: `Lanzador` (2/2)

```java
public class Lanzador {
    // main(...): en la diapositiva anterior

    /** Construye el comando que ejecuta la clase indicada con la misma JVM que este programa. */
    static List<String> comandoJava(String clase, String... argumentos) {
        List<String> comando = new ArrayList<>();
        comando.add(Path.of(System.getProperty("java.home"), "bin", "java").toString());
        comando.add("-cp");
        comando.add(System.getProperty("java.class.path"));
        comando.add(clase);
        for (String argumento : argumentos) {
            comando.add(argumento);
        }
        return comando;
    }
}
```

`ProcessBuilder` prepara el proceso, `start()` lo lanza y `Process` lo controla.

Cada argumento va en su propia cadena, y el hijo se lanza con la misma JVM que el padre.

A01.4 (1/2). Lanzar y vigilar un proceso hijo

1. Copia el ejemplo 6.3, compílalo y ejecútalo: `javac -d out *.java` y `java -cp out Lanzador`. Anota el comando, el PID del hijo, su resultado y su código
2. Cambia el intervalo a `"1", "3000000000"`, compila, ejecuta y déjalo en marcha
3. En **otra terminal**, `jps -l`: ¿cuál es el padre y cuál el hijo? ¿Por qué sale `jps`?
4. En *Detalles*, localiza los dos `java.exe`: ¿cuánta CPU usa cada uno?, ¿por qué el padre casi nada?, ¿cuántos hilos tienen?
5. `tasklist /fi "IMAGENAME eq java.exe"`: ¿coinciden los PID?

> **Tiempo:** 40 min, individual. **Entrega:** no se entrega; guarda las respuestas para el proyecto.

A01.4 (2/2). Lanzar y vigilar un proceso hijo

6. `jcmd <PID del hijo> VM.uptime` y `jcmd <PID del hijo> Thread.print`: ¿en qué estado está `main` y qué método ejecuta?
7. Baja la prioridad del hijo a *Por debajo de lo normal*: ¿notas algo?, ¿cuándo lo notarías?
8. Termina el hijo con `taskkill /pid <PID del hijo> /f`: ¿qué escribe el padre?, ¿qué código recibe?, ¿por qué no es 0?
9. **Si te da tiempo:** lanza `Lanzador` desde IntelliJ y páralo con *Stop*. ¿Sigue vivo el hijo (`jps -l`)? ¿Cómo se llama ese proceso?

> **Tiempo:** incluido en los 40 min. **Entrega:** no se entrega.

Terminar un proceso hijo

- `destroy()`: pide al proceso que termine; que sea normal o forzado depende del SO
- `destroyForcibly()`: lo termina a la fuerza, en el acto
- `isAlive()`: indica si sigue en marcha
- `supportsNormalTermination()`: si `destroy()` permite una terminación normal
- Para qué: tarda demasiado, ya no hace falta su resultado o el usuario cancela

Ejemplo 6.4: `Finalizar`

```java
import java.io.IOException;

public class Finalizar {
    public static void main(String[] args) throws IOException, InterruptedException {
        ProcessBuilder pb = new ProcessBuilder(comandoJava("ContadorPrimos", "1", "5000000000"));
        Process hijo = pb.start();
        System.out.println("Hijo lanzado con PID " + hijo.pid());
        Thread.sleep(2000);  // el padre espera 2 segundos sin hacer nada
        System.out.println("¿Sigue vivo? " + hijo.isAlive());
        System.out.println("Se pide al hijo que termine con destroy()");
        hijo.destroy();
        int codigo = hijo.waitFor();
        System.out.println("¿Sigue vivo? " + hijo.isAlive());
        System.out.println("Código de salida: " + codigo);
        System.out.println("¿El SO admite la terminación normal? " + hijo.supportsNormalTermination());
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Cada sistema lo termina a su manera

- Linux y macOS: `destroy()` envía `SIGTERM` y `destroyForcibly()`, `SIGKILL`
- Código de salida: 128 + la señal; 143 con `SIGTERM` y 137 con `SIGKILL`
- Windows: `destroy()` también termina a la fuerza, con otro código de salida
- **Un programa portable solo comprueba que el código no es `0`**
- `destroy()` no termina a los nietos: se recorren con `hijo.descendants()`
- Si se para el padre (*Stop* de IntelliJ), los hijos siguen: quedan **huérfanos**

`ProcessHandle`: cualquier proceso

- `ProcessHandle.current()` (el propio) y `ProcessHandle.of(pid)`
- `parent()`, `children()` (hijos directos), `descendants()` y `ProcessHandle.allProcesses()`
- `info()`: ejecutable, argumentos, usuario, inicio y tiempo de CPU
- Cada dato de `info()` es un `Optional`, porque el SO puede no darlo: se saca con `orElse(valor)`
- Desde un `Process`, con `hijo.toHandle()`

> Ejemplo 6.5 (`InfoProcesos`), en `codigo/ejemplo-6-5/`: consulta el propio proceso, su padre, sus hijos y cuántos procesos hay.

Proyecto A01.5: informe de ventas en paralelo

Una cadena tiene 8 tiendas. Os piden:

- Un **informe de ventas** por tienda y total, con **un proceso por tienda**
- Una **caja en directo**: una caja y un registro unidos por una **tubería**
- El hijo del informe, `ShopSummary`, os lo damos **hecho**; del resto, el **esqueleto**
- Proyecto Java **sin Maven** y con todas las clases **sin paquete**
- En grupo; una fase por sesión, y cada fase tiene que funcionar
- **Sin IA generativa**: el código lo escribís vosotros

El hijo que os damos: `ShopSummary`

Aspecto	Contrato
Uso	`java ShopSummary <tienda>`, de la 1 a la 8
Qué hace	Simula las ventas de la tienda: cada tienda da siempre lo mismo
Salida estándar	**Solo** `unidades;céntimos`, por ejemplo `150007132;42269946809`
Salida de error	Los mensajes de error
Código de salida	`0` si todo va bien; `1` si el argumento es incorrecto

Total esperado de las 8 tiendas: **1.200.014.349 uds** y **3.381.671.938,70 €**. El de cada tienda, en el enunciado.

Las clases que programáis

Clase	Papel	Qué hace
`SalesReport`	Padre	Lanza un `ShopSummary` por tienda y escribe el informe
`CashRegister`	Productor (la caja)	Escribe tickets `número;céntimos`, uno por línea
`Ledger`	Consumidor (el registro)	Lee los tickets, los suma y da el total
`LiveCashRegister`	Padre	Une la caja y el registro con una tubería

Cada clase viene en esqueleto: completáis sus `// TODO`, fase a fase.

Las fases del proyecto

Fase	Sesiones	Qué se añade	Modelo
1	S5	Lanzar un `ShopSummary`: PID y código de salida	Ejemplo 6.3
2	S6–7	Leer su resultado; un hijo por tienda y el total	Ejemplo 7.2a
3	S8	Tiempo límite y código de salida en el informe	Ejemplo 8.1b
4	S9	Caja en directo con `startPipeline()`, en dos escenarios	Ejemplo 8.4
5	S10 y S11–12	Uno a uno y todos a la vez, depuración y README	Ejemplo 9.1

Entrega: al final de la unidad. **Defensa individual:** en la última sesión.

A01.5, fase 1. Lanzar un hijo

1. Cread el proyecto en IntelliJ (Java, sin Maven) y copiad los `.java` de partida en `src/`
2. En el `main` de `SalesReport`, lanzad `ShopSummary` para la tienda 1 con `javaCommand`
3. El hijo hereda la entrada y la salida del padre con `inheritIO()`
4. El padre escribe el PID del hijo y su código de salida
5. Comprobad que sale el resultado de la tienda 1 del enunciado

> **Tiempo:** 30 min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad, en el Aula Virtual.

07

Comunicación entre procesos

La memoria está aislada: hacen falta mecanismos del sistema

Mecanismos de comunicación

Mecanismo	Sentido	Cuándo	En Java
Argumentos y entorno	Padre → hijo	Al lanzarlo	`ProcessBuilder`, `environment()`
Flujos estándar	Los dos	Mientras se ejecuta	`outputWriter`, `inputReader`…
Redirecciones	Según se configure	Mientras se ejecuta	`redirectInput`, `redirectOutput`…
Ficheros compartidos	Los dos	En cualquier momento	`Files`, `RandomAccessFile`
Tuberías	Uno escribe, otro lee	Mientras se ejecutan	`startPipeline`
Código de salida	Hijo → padre	Al terminar	`waitFor`, `exitValue`

Argumentos y variables de entorno

- **Argumentos**: van detrás del programa en el comando; el hijo los recibe en `String[] args`
- Cada cadena es **un** argumento, aunque tenga espacios: no hacen falta comillas
- **Entorno**: `pb.environment()` es un `Map` con una **copia** del entorno del padre
- Los cambios solo afectan al hijo, que lo lee con `System.getenv("NOMBRE")`
- Límites: solo texto, solo al lanzarlo y solo del padre al hijo

Ejemplo 7.1: `MostrarArgumentos` (el hijo)

```java
public class MostrarArgumentos {

    public static void main(String[] args) {
        System.out.println("Argumentos recibidos: " + args.length);
        for (int i = 0; i < args.length; i++) {
            System.out.println("  args[" + i + "] = \"" + args[i] + "\"");
        }
        System.out.println("Variable MODO = " + System.getenv("MODO"));
    }
}
```

Ejemplo 7.1: `PasarDatos` (el padre)

```
import java.io.IOException;
import java.util.Map;

public class PasarDatos {
    public static void main(String[] args) throws IOException, InterruptedException {
        ProcessBuilder pb = new ProcessBuilder(
                comandoJava("MostrarArgumentos", "primos.txt", "Hola mundo", "42"));

        Map<String, String> entorno = pb.environment();  // copia del entorno del padre
        entorno.put("MODO", "detallado");

        pb.inheritIO();
        Process hijo = pb.start();
        hijo.waitFor();
        System.out.println("En el padre, MODO = " + System.getenv("MODO"));
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Los tres flujos del hijo

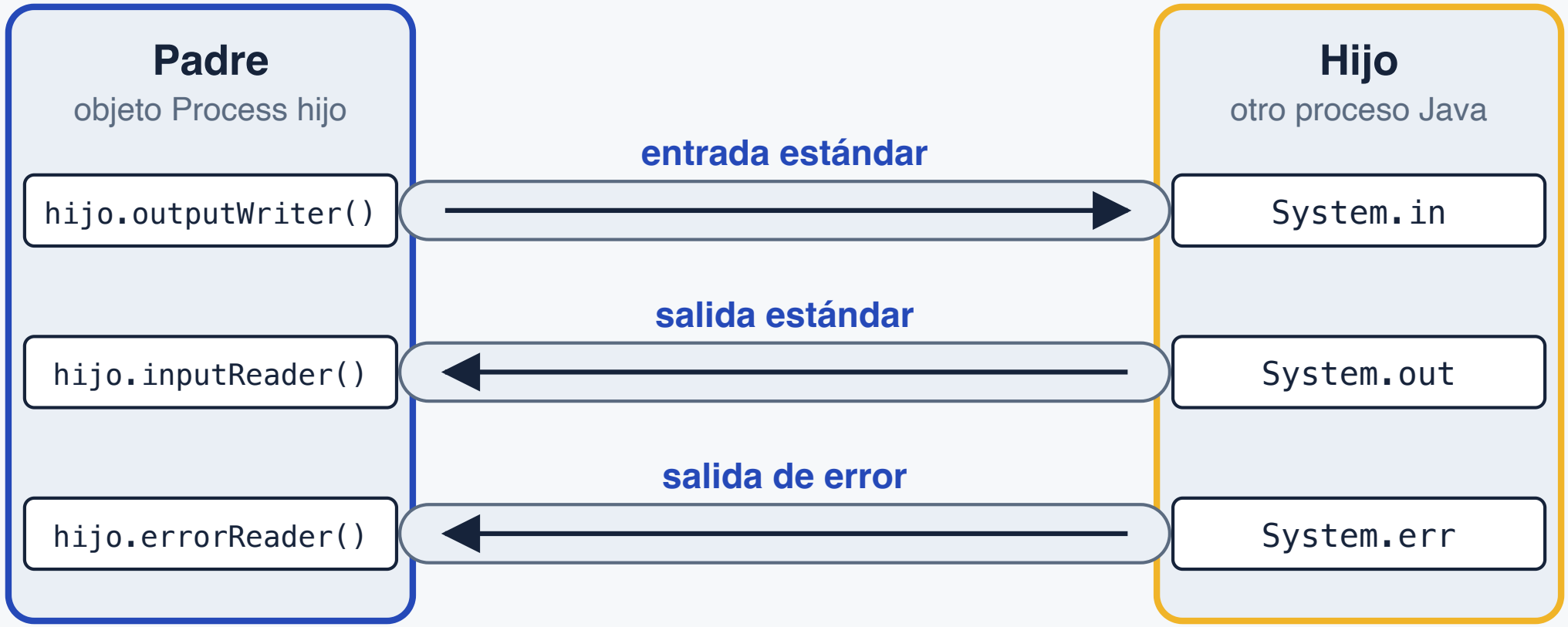


Figura 9. Cada flujo del hijo está unido al padre por una tubería. La salida del hijo es una entrada del padre

Los flujos desde el padre

Flujo del hijo	En el hijo	En el padre (Java 17+)	Forma clásica
Entrada estándar	`System.in`	`hijo.outputWriter()`	`hijo.getOutputStream()`
Salida estándar	`System.out`	`hijo.inputReader()`	`hijo.getInputStream()`
Salida de error	`System.err`	`hijo.errorReader()`	`hijo.getErrorStream()`

- Los nombres son **desde el punto de vista del padre**: la salida del hijo es su entrada
- `inputReader()` ya devuelve un `BufferedReader`; la forma clásica hay que envolverla

Ejemplo 7.2a: `LeerSalida`

```java
import java.io.BufferedReader;
import java.io.IOException;
public class LeerSalida {
    public static void main(String[] args) throws IOException, InterruptedException {
        Process hijo = new ProcessBuilder(comandoJava("ContadorPrimos", "1", "100000")).start();
        // La salida estándar del hijo es un flujo de entrada para el padre
        try (BufferedReader salidaHijo = hijo.inputReader()) {
            String linea = salidaHijo.readLine();
            long primos = Long.parseLong(linea);
            System.out.println("El hijo dice que hay " + primos + " primos hasta 100000");
            System.out.println("Eso es un " + (primos * 100.0 / 100000) + " % de los números");
        }
        int codigo = hijo.waitFor();
        System.out.println("Código de salida del hijo: " + codigo);
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 7.2b: `EscribirEntrada` (1/2)

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
public class EscribirEntrada {
    public static void main(String[] args) throws IOException, InterruptedException {
        Process hijo = new ProcessBuilder(comandoJava("FiltroPrimos")).start();
        // 1. Escribir en la entrada del hijo. Al cerrar el flujo, el hijo ve el fin de la entrada.
        try (BufferedWriter entradaHijo = hijo.outputWriter()) {
            String[] datos = {"10", "11", "12", "13", "hola", "97", "100"};
            for (String dato : datos) {
                entradaHijo.write(dato);
                entradaHijo.newLine();
            }
        }
        // 2. Leer su salida estándar y su salida de error: en la diapositiva siguiente
        System.out.println("Código de salida: " + hijo.waitFor());
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 7.2b: `EscribirEntrada` (2/2)

```
        // 2. Leer su salida estándar (los primos) y su salida de error (los avisos)
        System.out.println("Primos que devuelve el hijo:");
        try (BufferedReader salidaHijo = hijo.inputReader()) {
            String linea;
            while ((linea = salidaHijo.readLine()) != null) {
                System.out.println("  " + linea);
            }
        }
        System.out.println("Errores que escribe el hijo:");
        try (BufferedReader errorHijo = hijo.errorReader()) {
            String linea;
            while ((linea = errorHijo.readLine()) != null) {
                System.out.println("  " + linea);
            }
        }
```

Cuidado con los búferes

Las tuberías entre el padre y el hijo tienen **capacidad limitada**.

- Si el padre llama a `waitFor()` antes de leer y el hijo escribe mucho, la tubería se llena
- El hijo se bloquea esperando a que alguien lea; el padre, esperando a que el hijo termine
- **Interbloqueo**: nadie avanza, y no hay ningún error
- Para evitarlo: leer las salidas **antes** de `waitFor()`…
- …unir la de error a la estándar con `redirectErrorStream(true)` o redirigirlas (apartado 7.3)

Lee la salida del hijo antes de `waitFor()` y cierra su entrada cuando acabes de escribir.

Si no, el padre y el hijo pueden quedarse esperándose el uno al otro para siempre.

A01.5, fase 2 (1/2). Argumentos y salida estándar

1. El padre ya no usa `inheritIO()`: pasa la tienda a `ShopSummary` **por argumento**
2. El padre **lee el resultado** de la salida estándar del hijo
3. Separa las unidades y los céntimos con `split(";")`
4. Escribe las unidades y el importe en euros
5. Comprobad que el resultado de la tienda 1 coincide con el del enunciado

> **Tiempo:** 35 min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad.

Redirecciones

Método de `ProcessBuilder`	Efecto	Terminal
`redirectInput(File)`	El hijo lee de un fichero	`< f.txt`
`redirectOutput(File)`	La salida va a un fichero (lo sobrescribe)	`> f.txt`
`redirectOutput(Redirect.appendTo(File))`	Igual, pero añade al final	`>> f.txt`
`redirectError(File)`	La salida de error va a un fichero	`2> f.txt`
`redirectErrorStream(true)`	La de error se une a la estándar	`2>&1`
`inheritIO()`	Los tres flujos, los de la consola del padre	—

Ejemplo 7.3: `Redirecciones` (1/2)

```java
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
public class Redirecciones {
    public static void main(String[] args) throws IOException, InterruptedException {
        File numeros = new File("numeros.txt");
        File primos = new File("primos.txt");
        File errores = new File("errores.log");
        Files.writeString(numeros.toPath(), "2\n3\n4\ncinco\n7\n8\n9\n11\n");
        ProcessBuilder pb = new ProcessBuilder(comandoJava("FiltroPrimos"));
        pb.redirectInput(numeros);                                  // entrada desde fichero
        pb.redirectOutput(primos);                                  // salida a fichero (lo sobrescribe)
        pb.redirectError(ProcessBuilder.Redirect.appendTo(errores));// error: se añade al final
        int codigo = pb.start().waitFor();
        // Mostrar el código de salida y los dos ficheros: en la diapositiva siguiente
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 7.3: `Redirecciones` (2/2)

```
        int codigo = pb.start().waitFor();
        System.out.println("Código de salida: " + codigo);
        System.out.println("--- primos.txt ---");
        System.out.print(Files.readString(primos.toPath()));
        System.out.println("--- errores.log ---");
        System.out.print(Files.readString(errores.toPath()));
```

- Equivale a `java FiltroPrimos < numeros.txt > primos.txt 2>> errores.log`
- Aquí sí se puede llamar a `waitFor()` enseguida: no hay tuberías hacia el padre que se llenen

Ficheros compartidos

- Uno escribe y otro lee: sencillo, admite muchos datos y se puede revisar a mano
- Esquema habitual: **cada hijo deja su resultado en su propio fichero** y el padre los combina al final
- Para saber cuándo ha terminado de escribir el otro: `waitFor()`
- Si varios escriben **a la vez en el mismo fichero**, los datos se corrompen (apartado 8.2)

Tuberías entre procesos

Canal **unidireccional** del SO: uno escribe por un extremo y otro lee por el otro, en orden (FIFO).

- Es el `|` de la terminal: `java GeneradorNumeros 1 20 | java FiltroPrimos`
- Si está **vacía**, el que lee se bloquea hasta que haya datos
- Si está **llena**, el que escribe se bloquea hasta que haya sitio (en Linux, 64 KiB)
- Cuando el que escribe cierra su extremo, el que lee recibe el **fin de la entrada**
- `ProcessBuilder.startPipeline(lista)` lanza varios procesos unidos por tuberías (Java 9)

Una tubería de tres procesos



Cada proceso lee su entrada estándar
y escribe en su salida estándar.

Los tres trabajan a la vez.

Figura 10. Ejemplo 7.5: GeneradorNumeros, FiltroPrimos y ContarLineas cuentan los primos hasta un millón

Ejemplo 7.5: `Tuberia` (1/2)

```java
import java.io.BufferedReader;
import java.io.IOException;

public class Tuberia {
    public static void main(String[] args) throws IOException, InterruptedException {
        List<ProcessBuilder> etapas = new ArrayList<>();
        etapas.add(new ProcessBuilder(comandoJava("GeneradorNumeros", "1", "1000000")));
        etapas.add(new ProcessBuilder(comandoJava("FiltroPrimos")));
        etapas.add(new ProcessBuilder(comandoJava("ContarLineas")));

        long inicio = System.currentTimeMillis();
        List<Process> procesos = ProcessBuilder.startPipeline(etapas);
        // Mostrar los PID, leer el resultado y esperar a todos: en la diapositiva siguiente
        System.out.println("Tiempo: " + (System.currentTimeMillis() - inicio) + " ms");
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 7.5: `Tuberia` (2/2)

```
        for (Process p : procesos) {
            System.out.println("Proceso de la tubería con PID " + p.pid());
        }

        // Solo se lee la salida del último proceso de la cadena
        Process ultimo = procesos.get(procesos.size() - 1);
        try (BufferedReader salida = ultimo.inputReader()) {
            System.out.println("Primos entre 1 y 1000000: " + salida.readLine());
        }
        for (Process p : procesos) {
            p.waitFor();
        }
```

A01.5, fase 2 (2/2). Un hijo por tienda y el total

1. La salida de error del hijo se hereda (`redirectError`): sus mensajes salen en la consola
2. El padre lanza **un hijo por cada tienda**, de la 1 a la 8 (constante `SHOPS`)
3. Escribe una línea por tienda y el total
4. Comprobad el total con el del enunciado

> **Tiempo:** 40 min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad.

08

Sincronización entre procesos

Esperar a otro proceso y no estorbarse

Sincronizar procesos

Sincronizar es coordinar procesos concurrentes para que respeten un orden o no se estorben.

- **Condición de sincronización**: un proceso espera a que ocurra algo que depende de otro
  - El padre no puede usar el resultado hasta que el hijo termina
  - El consumidor no puede procesar un dato hasta que el productor lo envía
- **Exclusión mutua**: varios procesos no deben usar a la vez un recurso compartido

Esperar a un hijo

Método	Qué hace
`waitFor()`	Bloquea al padre hasta que el hijo termina y devuelve su código de salida
`waitFor(tiempo, unidad)`	Espera como mucho ese tiempo: `true` si ha terminado, `false` si no
`exitValue()`	El código de un hijo que ya ha terminado; si no, `IllegalThreadStateException`
`isAlive()`	Si sigue en marcha, sin esperar

Mientras espera en `waitFor()`, el padre está **bloqueado**: no consume CPU.

El código de salida

Es la forma que tiene el hijo de decir cómo le ha ido: **`0` es éxito** y cualquier otro valor, error.

- `main` termina normalmente → `0`
- `System.exit(n)` → `n` (en Unix, solo de `0` a 255)
- Excepción no capturada → 1
- Terminado por una señal (Unix) → 128 + la señal
- Cada hijo define y documenta los suyos: `ContadorPrimos` usa `0`, 1 y 2

Ejemplo 8.1b: `TiempoLimite` (1/2)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.util.concurrent.TimeUnit;
public class TiempoLimite {
    public static void main(String[] args) throws IOException, InterruptedException {
        contarConLimite("100000");        // termina enseguida
        contarConLimite("5000000000");    // tardaría minutos
    }

    static void contarConLimite(String hasta) throws IOException, InterruptedException {
        Process hijo = new ProcessBuilder(comandoJava("ContadorPrimos", "1", hasta)).start();
        long inicio = System.currentTimeMillis();
        boolean terminado = hijo.waitFor(3, TimeUnit.SECONDS);
        long espera = System.currentTimeMillis() - inicio;
        // Si ha terminado, leer su resultado; si no, terminarlo: en la diapositiva siguiente
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 8.1b: `TiempoLimite` (2/2)

```
        if (terminado) {
            try (BufferedReader salida = hijo.inputReader()) {
                System.out.println("Hasta " + hasta + ": " + salida.readLine()
                        + " primos (esperados " + espera + " ms)");
            }
        } else {
            System.out.println("Hasta " + hasta + ": sin respuesta en " + espera
                    + " ms; se termina el proceso " + hijo.pid());
            hijo.destroyForcibly();
            hijo.waitFor();
            System.out.println("  Código de salida tras destroyForcibly(): " + hijo.exitValue());
        }
```

Condición de carrera

El resultado depende del **orden** en que se intercalan las operaciones de los procesos.

- «Sumar 1» parece una sola operación, pero son tres pasos: **leer, sumar y escribir**
- Otro proceso puede meterse entre ellos
- Ejemplo 8.2: 10 procesos suman 1, diez mil veces cada uno, al número guardado en un fichero
- Debería dar 100.000 y da **18.970**; y en cada ejecución, un número distinto

Ejemplo 8.2: `Incrementador`

```java
import java.io.IOException;
import java.io.RandomAccessFile;
public class Incrementador {
    public static void main(String[] args) throws IOException {
        String fichero = args[0];
        int veces = Integer.parseInt(args[1]);

        try (RandomAccessFile contador = new RandomAccessFile(fichero, "rw")) {
            byte[] texto = new byte[10];
            for (int i = 0; i < veces; i++) {
                contador.seek(0);
                contador.readFully(texto);                          // 1. leer
                long valor = Long.parseLong(new String(texto));
                valor = valor + 1;                                  // 2. sumar
                contador.seek(0);
                contador.write(String.format("%010d", valor).getBytes());  // 3. escribir
            }
        }
    }
}
```

¿Dónde se pierde la suma?

Instante	Proceso P1	Proceso P2	En el fichero
1	Lee 41		41
2		Lee 41	41
3	Suma: 42		41
4	Escribe 42		42
5		Suma: 42	42
6		Escribe 42	42

Los dos han sumado 1, pero el valor solo ha subido 1: la suma de P1 se ha perdido.

Sección crítica y exclusión mutua

- **Sección crítica**: el código que usa el recurso compartido; aquí, leer, sumar y escribir
- **Exclusión mutua**: como mucho un proceso dentro de la sección crítica a la vez
- **Operación atómica**: se ejecuta de una vez, sin que nadie intervenga a medias
- **Condición de competencia**: varios procesos necesitan un recurso de forma exclusiva
- **Espera activa**: comprobar en un bucle si ya se puede seguir, gastando CPU
- **Espera con bloqueo**: el SO despierta al proceso cuando ocurre lo que espera; no gasta CPU

Interbloqueo e inanición

Interbloqueo: cada proceso espera algo que solo le puede dar otro del grupo, y ninguno avanza. Hacen falta las cuatro condiciones:

1. **Exclusión mutua**: los recursos no se pueden compartir
2. **Retención y espera**: retiene recursos mientras espera otros
3. **No expropiación**: no se le puede quitar un recurso
4. **Espera circular**: cada uno espera un recurso del siguiente

Inanición: un proceso espera siempre porque otros pasan delante; **el resto sí avanza**.

Semáforos

Un entero no negativo con una cola de procesos bloqueados (Dijkstra, 1965).

- `wait`: si vale más de 0, lo decrementa y sigue; si vale 0, se bloquea
- `signal`: despierta a un bloqueado o, si no hay, lo incrementa
- **Binario** (vale 1): exclusión mutua. **Contador** (vale *n*): *n* recursos

```
semáforo mutex = 1
wait(mutex)
    leer el contador, sumar 1, escribirlo      ← sección crítica
signal(mutex)
```

Monitores y paso de mensajes

Monitor

- Objeto que agrupa los datos compartidos y sus operaciones
- Él mismo garantiza que solo un proceso las ejecuta a la vez
- Variables de condición para esperar y avisar
- En Java: `synchronized` y `ReentrantLock` (UD 2)

Paso de mensajes

- Sin memoria compartida: **enviar** y **recibir** por un canal
- Recibir bloquea hasta que llega un mensaje
- Si el canal está lleno, enviar bloquea
- En Java: tuberías (aquí), `BlockingQueue` (UD 2), sockets (UD 3)

Que una ejecución salga bien no demuestra que no haya condición de carrera.

El error depende de los tiempos: puede aparecer solo algunas veces y desaparecer al depurar.

A01.6. La condición de carrera

1. Ejecutad **tres veces** el ejemplo 8.2 (`Carrera` e `Incrementador`): ¿sale siempre lo mismo?
2. Cambiad `PROCESOS` a 1 y después a 2: ¿se pierden sumas?
3. Con 10 procesos, añadid `Thread.sleep(300);` después de `hijos.add(pb.start());`. ¿Está arreglado? Probad con 20 ms
4. Escribid un intercalado de P1 y P2, empezando en 7, en el que se pierda una suma
5. Señalad la **sección crítica** y protegedla con un **semáforo**, en pseudocódigo
6. En la caja en directo del proyecto, ¿por qué no hay condición de carrera en la tubería?

> **Tiempo:** 20 min, en grupo. **Entrega:** no se entrega; se ponen en común las preguntas 3 y 6.

A01.5, fase 3. Tiempo límite y código de salida

1. El padre espera a cada hijo **con un tiempo límite**
2. Si no ha terminado a tiempo, lo termina
3. Si el **código de salida** no es `0`, no lee su resultado
4. El informe dice qué tiendas han fallado y con qué código
5. Probadlo con `SHOPS = 9` (la tienda 9 no existe) y con `TIMEOUT = 0`

> **Tiempo:** 25 min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad.

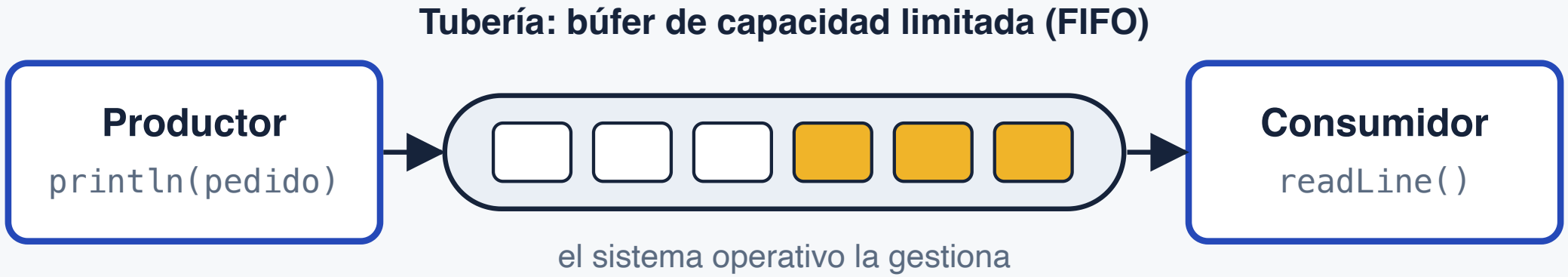
El productor y el consumidor

Un **productor** genera datos y un **consumidor** los procesa, a velocidades distintas, con un **búfer** limitado entre los dos. Hay que garantizar:

1. El consumidor espera si el búfer está **vacío**
2. El productor espera si el búfer está **lleno**
3. Los dos no corrompen el búfer al usarlo a la vez

Entre procesos, una **tubería** resuelve las tres. Es paso de mensajes: cada línea es un mensaje.

La tubería como búfer



Tubería llena

println() bloquea al productor

hasta que el consumidor lee

Tubería vacía

readLine() bloquea al consumidor

hasta que el productor escribe

Figura 11. Nadie programa las esperas: el sistema operativo bloquea a cada proceso cuando hace falta

Ejemplo 8.4: Productor

```java
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
public class Productor {
    static final DateTimeFormatter HORA = DateTimeFormatter.ofPattern("HH:mm:ss.SSS");

    public static void main(String[] args) throws InterruptedException {
        int pedidos = Integer.parseInt(args[0]);
        int pausa = Integer.parseInt(args[1]);
        int tamano = Integer.parseInt(args[2]);
        for (int i = 1; i <= pedidos; i++) {
            Thread.sleep(pausa);
            String pedido = "pedido " + i + " ";
            String relleno = ".".repeat(Math.max(0, tamano - pedido.length() - 1));
            System.out.println(pedido + relleno);  // se bloquea si la tubería está llena
            System.err.println(LocalTime.now().format(HORA) + " Productor  escribe pedido " + i);
        }
        System.err.println(LocalTime.now().format(HORA) + " Productor  termina");
    }
}
```

Ejemplo 8.4: `Consumidor` (1/2)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
public class Consumidor {
    static final DateTimeFormatter HORA = DateTimeFormatter.ofPattern("HH:mm:ss.SSS");

    public static void main(String[] args) throws IOException, InterruptedException {
        int pausa = Integer.parseInt(args[0]);
        BufferedReader entrada = new BufferedReader(new InputStreamReader(System.in));
        int recibidos = 0;
        String linea;
        // Bucle que lee los pedidos: en la diapositiva siguiente
        System.err.println(LocalTime.now().format(HORA) + " Consumidor fin de la entrada: "
                + recibidos + " pedidos");
    }
}
```

Ejemplo 8.4: `Consumidor` (2/2)

```
while ((linea = entrada.readLine()) != null) {  // se bloquea si la tubería está vacía
    recibidos++;
    String[] partes = linea.split(" ");
    System.err.println(LocalTime.now().format(HORA) + " Consumidor lee " + partes[0] + " " + partes[1]);
    Thread.sleep(pausa);  // simula el tiempo de procesar el pedido
}
```

- `readLine()` bloquea al consumidor mientras la tubería está vacía
- Devuelve `null` cuando el productor ha terminado y ya no quedan pedidos

Ejemplo 8.4: `ProductorConsumidor` (1/2)

```
import java.io.IOException;

public class ProductorConsumidor {
    public static void main(String[] args) throws IOException, InterruptedException {
        System.out.println("== Escenario 1: productor lento (500 ms), consumidor rápido ==");
        ejecutar("5", "500", "100", "0");

        System.out.println("== Escenario 2: productor rápido, consumidor lento (300 ms) ==");
        ejecutar("10", "0", "16384", "300");
    }
    // ejecutar(...): en la diapositiva siguiente
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 8.4: `ProductorConsumidor` (2/2)

```
    static void ejecutar(String pedidos, String pausaProductor, String tamano, String pausaConsumidor)
            throws IOException, InterruptedException {
        ProcessBuilder productor = new ProcessBuilder(
                comandoJava("Productor", pedidos, pausaProductor, tamano));
        ProcessBuilder consumidor = new ProcessBuilder(comandoJava("Consumidor", pausaConsumidor));

        // Los dos escriben sus anotaciones en la salida de error, que se ve en la consola del padre
        productor.redirectError(ProcessBuilder.Redirect.INHERIT);
        consumidor.redirectError(ProcessBuilder.Redirect.INHERIT);

        List<Process> procesos = ProcessBuilder.startPipeline(List.of(productor, consumidor));
        for (Process p : procesos) {
            p.waitFor();
        }
    }
```

Quién espera a quién

Escenario 1

- Productor lento, consumidor rápido
- La tubería está casi siempre **vacía**
- El consumidor espera bloqueado en `readLine()`, sin gastar CPU
- Lee cada pedido en el mismo milisegundo en que se escribe

Escenario 2

- Productor rápido, consumidor lento
- Pedidos de 16 KiB: la tubería se **llena**
- El productor escribe 5 de golpe y después se bloquea en `println`
- Termina antes: los pedidos que faltan esperan en la tubería

> **Con una tubería, el sistema operativo hace las esperas: bloquea al que lee si está vacía y al que escribe si está llena.**

Con hilos, el mismo problema se resuelve con una `BlockingQueue` (UD 2).

A01.5, fase 4. La caja en directo

1. `CashRegister` y `Ledger`: adaptad el productor y el consumidor del ejemplo 8.4
2. `LiveCashRegister.runScenario` los une con `startPipeline()` y espera a los dos
3. Trazas con la hora (ms) en la salida de error, visibles en la consola del padre
4. **Escenario 1:** 5 tickets; pausas: caja `500` ms, registro `0`
5. **Escenario 2:** `10` tickets; pausas: caja `0`, registro `300` ms
6. Con las trazas, averiguad **quién espera a quién** en cada escenario

> **Tiempo:** `40` min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad.

09

Aplicaciones multiproceso

Repartir un trabajo entre varios procesos

Repartir un trabajo entre procesos

1. **Descomponer** el problema en partes independientes: intervalos, ficheros, filas...
2. **Decidir cuántos procesos**:
   `Runtime.getRuntime().availableProcessors()`
3. **Lanzar todos antes de esperar a ninguno**; si no, se trabaja en secuencia
4. **Recoger** el resultado de cada hijo y comprobar su código de salida
5. **Combinar** los resultados parciales en el resultado final

Ejemplo 9.1: `RepartoPrimos` (1/4)

```java
import java.io.BufferedReader;
import java.io.IOException;
public class RepartoPrimos {
    static final long HASTA = 100_000_000L;

    // main(...): mide contarEnParalelo con 1, 2, 4 procesos y tantos como núcleos (2/4)

    /** Reparte [1, HASTA] en tramos iguales, lanza un hijo por tramo y suma sus resultados. */
    static long contarEnParalelo(int procesos, boolean mostrarTramos)
            throws IOException, InterruptedException {
        List<Process> hijos = new ArrayList<>();
        long tramo = HASTA / procesos;
        // 1. Lanzar todos los hijos, sin esperar a ninguno (3/4)
        // 2. Recoger el resultado de cada hijo y comprobar que ha terminado bien (4/4)
        return total;
    }
    // comandoJava(...) y sus imports (Path, ArrayList, List): los del ejemplo 6.3
}
```

Ejemplo 9.1: `RepartoPrimos` (2/4)

```
    public static void main(String[] args) throws IOException, InterruptedException {
        int nucleos = Runtime.getRuntime().availableProcessors();
        System.out.println("Núcleos disponibles: " + nucleos);
        System.out.println("Primos entre 1 y " + HASTA);
        int[] configuraciones = {1, 2, 4, nucleos};
        long tiempoConUno = 0;
        for (int procesos : configuraciones) {
            long inicio = System.currentTimeMillis();
            long total = contarEnParalelo(procesos, procesos == nucleos);
            long tiempo = System.currentTimeMillis() - inicio;
            if (procesos == 1) {
                tiempoConUno = tiempo;
            }
            System.out.printf("%2d proceso(s): %d primos en %5d ms (aceleración %.2f)%n",
                    procesos, total, tiempo, (double) tiempoConUno / tiempo);
        }
    }
```

Ejemplo 9.1: `RepartoPrimos` (3/4)

```
        // 1. Lanzar todos los hijos, sin esperar a ninguno
        for (int i = 0; i < procesos; i++) {
            long desde = i * tramo + 1;
            long hasta = (i == procesos - 1) ? HASTA : (i + 1) * tramo;
            ProcessBuilder pb = new ProcessBuilder(
                    comandoJava("ContadorPrimos", String.valueOf(desde), String.valueOf(hasta)));
            pb.redirectError(ProcessBuilder.Redirect.INHERIT);
            Process hijo = pb.start();
            hijos.add(hijo);
            if (mostrarTramos) {
                System.out.println("   PID " + hijo.pid() + ": de " + desde + " a " + hasta);
            }
        }
```

Ejemplo 9.1: `RepartoPrimos` (4/4)

```
// 2. Recoger el resultado de cada hijo y comprobar que ha terminado bien
long total = 0;
for (Process hijo : hijos) {
    try (BufferedReader salida = hijo.inputReader()) {
        total += Long.parseLong(salida.readLine());
    }
    int codigo = hijo.waitFor();
    if (codigo != 0) {
        throw new IllegalStateException(
        "El proceso " + hijo.pid() + " ha fallado con código " + codigo);
    }
}
return total;
```

- Se lee el resultado **antes** de `waitFor()`
- Si un hijo falla, se detiene: el total estaría mal

¿Por qué no va 10 veces más rápido?

Procesos	1	2	4	10
Tiempo (ms)	13.976	9.058	5.098	3.465
Aceleración	1,00	1,54	2,74	4,03

- El resultado es **el mismo** siempre (5.761.455 primos): es la primera prueba
- **Reparto desigual**: los números grandes cuestan más, y manda el tramo más lento
- **Núcleos distintos**: 4 de rendimiento y 6 de eficiencia, más lentos
- **Parte secuencial y coste fijo**: arrancar cada JVM y recoger de uno en uno (Amdahl)

Vigilar la aplicación mientras se ejecuta

- **Desde el programa**: el PID de cada hijo, `isAlive()` y `ProcessHandle`
- **Administrador de tareas**: un `java.exe` por proceso en *Detalles*; los núcleos ocupados en *Rendimiento*
- `jps -l`: qué clase ejecuta cada proceso Java
- El padre casi no usa CPU: está bloqueado en `readLine()`, esperando el resultado
- Cada hijo usa un núcleo entero: con 8 procesadores lógicos, un 12 % aproximadamente

A01.5, fase 5 (1/2). Uno a uno y todos a la vez

1. `SalesReport` hace el informe **dos veces** en la misma ejecución
2. Primero **uno a uno**: lanza cada hijo cuando ha terminado el anterior
3. Después **todos a la vez**: lanza todos los hijos antes de esperar a ninguno
4. Escribe el tiempo de cada modo y cuántas veces más rápido va el segundo
5. **Ampliación:** demostrad que la caja se bloquea cuando la tubería se llena

> **Tiempo:** 45 min en clase, en grupo. **Entrega:** con el proyecto, al final de la unidad.

Depurar con IntelliJ IDEA

1. **Punto de ruptura**: clic en el margen, junto al número de línea (Ctrl+F8)
2. **Depurar**: botón *Debug* (Mayús+F9); el programa se detiene en el punto de ruptura
3. **Examinar**: la vista *Variables* y *Evaluate Expression* (Alt+F8), por ejemplo `hijo.isAlive()`
4. **Avanzar**: *Step Over* (F8), *Step Into* (F7), *Step Out* (Mayús+F8) y *Resume* (F9)

Depurar una aplicación multiproceso

- El depurador **solo controla al padre**: un punto de ruptura en `ContadorPrimos` no detiene nada
- Mientras el padre está detenido, **los hijos siguen trabajando**
- Detener el padre con *Stop* deja a los hijos **huérfanos**
- Útil: examinar `pb.command()` y ejecutarlo a mano en una terminal
- Útil: detenerse después de `waitFor()` y mirar el código de salida
- Útil: comprobar en `hijos` que se han lanzado todos antes de esperar

El README de una aplicación multiproceso

Sección	Qué cuenta
Descripción	Qué hace, en dos o tres líneas
Requisitos	La versión de Java y lo demás que haga falta
Cómo se ejecuta	Qué clase se lanza, desde IntelliJ y desde la terminal
Procesos	Cada clase: padre o hijo, qué recibe, qué devuelve y sus códigos de salida
Pruebas realizadas	Qué se ha probado, qué se esperaba y qué ha salido
Limitaciones conocidas	Lo que no funciona o se podría mejorar

La tabla de procesos: el contrato de cada uno

Clase	Papel	Entrada	Salida
`RepartoPrimos`	Padre: divide el intervalo, lanza un hijo por tramo y suma	—	Informe por la consola
`ContadorPrimos`	Hijo: cuenta los primos de su tramo	Argumentos `<desde> <hasta>`	Solo el número de primos, en la salida estándar

- Es la sección más importante: lo que necesita saber quien lanza cada hijo
- Se completa con un comentario al principio de cada hijo: su uso y sus códigos de salida

Lanza todos los hijos antes de esperar a ninguno.

Si esperas a cada uno en el mismo bucle que los lanza, el programa funciona, pero en secuencia.

A01.5, fase 5 (2/2). Depurar y documentar

1. **Depurad** el padre: punto de ruptura tras esperar a un hijo; su código en *Variables*. Captura
2. Completad el **README** con la plantilla
3. `A01-5-grupoN.zip`: los `.java` (también `ShopSummary.java`) y el README, sin `out/`
4. **Defensa individual:** 2-3 minutos sobre **cualquier parte** del proyecto

> **Tiempo:** la última sesión y lo que falte, en casa, en grupo. **Entrega:** al final de la unidad, en el Aula Virtual.

Resumen (1/2)

- Un **proceso** es un programa en ejecución: PID, memoria aislada, estado y recursos, en el PCB
- El SO los alterna con **multitarea expulsiva**; estados: nuevo, listo, en ejecución, bloqueado y terminado
- El **planificador** elige (FIFO, SJF, SRT, Round Robin, prioridades); se mide el retorno, la espera y la respuesta
- Un **hilo** es una secuencia de ejecución dentro de un proceso; los hilos comparten su memoria
- **Concurrente**: tareas que se solapan. **Paralela**: a la vez en una máquina. **Distribuida**: varias máquinas en red

Resumen (2/2)

- `start()` lanza, `Process` controla y `ProcessHandle` consulta
- Argumentos, entorno, flujos, redirecciones, ficheros y **tuberías**
- `waitFor()`, con o sin límite, y el **código de salida** (`0`, éxito)
- Carrera, interbloqueo, inanición y **productor-consumidor**
- Lanzar todos, recoger, comprobar, depurar y documentar
- Próxima unidad: **UD 2. Programación multihilo**